```
---
title: "Modulo_2"
output:
  pdf_document: default
  html_document: default
date: "2026-04-01"
---

```{r setup, include=FALSE}
knitr::opts_chunk$set(echo = TRUE)
```


##Ejercicio 1 de algoritmos

Consigna: Las ultimas 3 cifras de mi DNI son 506, consigna crear una variable que tenga ese numero

```{r}
DNI=506
```

Consigna: Crear un vector que tenga todos los numeros enteros del 1 al 506

#1.4 GENERAR UN VECTOR SECUENCIA

```{r}
secuencia_DNI<-(1:506)
```

##Ejercicio 2

Calcular la suma de todos los valores del vector secuencia_DNI

```{r}
total<-0
valorfinal<-length(secuencia_DNI)
for (i in 1:valorfinal) {
total<-total+i
total
}
total
```

1.5 SERIE DE FIBONACHI

```{r}
# Definimos la cantidad de elementos basados en tu variable DNI
n<-506
fibonacci <- numeric(n)

# Inicializamos los dos primeros valores
fibonacci[1] <- 0
fibonacci[2] <- 1

# Generamos la serie mediante un bucle for
for (i in 3:n) {
  fibonacci[i] <- fibonacci[i-1] + fibonacci[i-2]
}

# Mostramos el resultado
print(fibonacci)
```
```

1.6 DEFINICION MATEMATICA RECURRENTE

```{r}
f0<-0   #Valores iniciales
f1<-1
f2<-0
i<-0  #Número de iteraciones incial
valorfinal<-1000000 #Umbral

while (f2<valorfinal) {

  f2=f0+f1
  f0=f1
  f1=f2
  i<-i+1

}
f2
i
```

1.7 Comparacion del método burbuja y el método sort de R

```{r}
library(microbenchmark)
library(ggplot2)

# 1. Definimos la función de Burbuja
bubble_sort <- function(x) {
  n <- length(x)
  for (i in 1:(n - 1)) {
    for (j in 1:(n - i)) {
      if (x[j] > x[j + 1]) {
        temp <- x[j]
        x[j] <- x[j + 1]
        x[j + 1] <- temp
      }
    }
  }
  return(x)
}

# 2. Creamos la muestra de 20,000 números aleatorios
set.seed(506) # Usando tu DNI como semilla
muestra_grande <- runif(20000)

# 3. Ejecutamos el microbenchmark
# Nota: Reducimos 'times' a 1 porque Burbuja es muy lento con 20k elementos
comparativa_20k <- microbenchmark(
  Burbuja = bubble_sort(muestra_grande),
  Nativo_R = sort(muestra_grande),
  times = 1
)

# 4. Ver los resultados en consola
print(comparativa_20k)

# 5. Gráfico de Violín para visualizar la diferencia
autoplot(comparativa_20k) +
  labs(title = "Burbuja vs Sort Nativo (N = 20,000)",
       subtitle = "Escala logarítmica de tiempo",
       y = "Tiempo de ejecución",
       x = "Algoritmo") +
  theme_minimal()
```

```
##Ejercicio 3

Repetir el ejercicio anterior pero en Python

```{python}

suma = 0

for i in range(1, 507):
    suma+=i
print(suma)
```


##Ejercicio 4
¿Cuanto tarda en correr el ejercicio 2?


```{r}
inicio<-Sys.time()

total<-0
valorfinal<-10000000*length(secuencia_DNI)
for (i in 1:valorfinal) {
total<-total+i
total
}
total

final<-Sys.time()
final-inicio
```

Ejercicio 5

Repetir el ejercicio anterior utilizando la cabeza de Gauss

```{r}
secuencia_DNI<-(1:506)
valorfinal <- 1e8 * length(secuencia_DNI)

inicio1 <- Sys.time()

total1 <- 0
for (i in 1:valorfinal) {
  total1 <- total1 + i
}

final1 <- Sys.time()
tiempo1 <- final1 - inicio1

cat("Resultado 1:", total1, "\n")
cat("Tiempo Fuerza Bruta:", tiempo1, "segundos\n")

```

Ejercicio 6

1.7.2 Ordenamiento burbuja

```{r}
# --- Script de Muestreo y Ordenamiento en R ---
```
```

```
# 1. Definir la muestra de 100 números (del 1 al 100)
poblacion <- 1:100

# 2. Tomar 10 números aleatorios de esa muestra
# set.seed() es opcional, sirve para que los números no cambien cada vez que corres el código
# set.seed(123)

muestra_aleatoria <- sample(poblacion, 10)

# 3. Ordenar los números de menor a mayor
muestra_ordenada <- sort(muestra_aleatoria)

# --- MOSTRAR RESULTADOS EN CONSOLA ---

cat("1. Muestra de 10 números aleatorios (sin orden):\n")
print(muestra_aleatoria)

cat("\n2. Muestra ordenada de menor a mayor:\n")
print(muestra_ordenada)
```

1.8 La penitencia de Newton

```{r}
library(microbenchmark)

# Desactivar notación científica
options(scipen = 999)

n <- 1010

# Algoritmo A: Fuerza Bruta (El "castigo" del profesor)
suma_bucle <- function(n) {
  resultado <- 0
  for (i in 1:n) {
    resultado <- resultado + i
  }
  return(resultado)
  resultado
}

# Algoritmo B: Fórmula Matemática (La "astucia" de Newton/Gauss)
suma_formula <- function(n) {
  return((n * (n + 1)) / 2)
  suma_formula
}

# Verificación de eficiencia
comparativa_newton <- microbenchmark(
  Fuerza_Bruta = suma_bucle(n),
  Formula_Gauss = suma_formula(n),
  times = 10
)

print(comparativa_newton)
```

2 Agrupacion K-mean

Medición de Performance: Agrupamiento K-MeansEn el análisis de datos industriales, no basta con agrupar los componentes; es imperativo evaluar qué tan "buenas" son esas agrupaciones. A diferencia del aprendizaje supervisado, donde comparamos predicciones contra etiquetas reales, en el Clustering medimos la calidad mediante la cohesión interna y la separación externa.1. Métricas de Evaluación InternaPara determinar si la segmentación de maquinaria tiene sentido estadístico, se utilizan principalmente dos indicadores:WSS (Within-Cluster Sum of Squares): Mide la suma de los cuadrados dentro de cada grupo. Representa la variabilidad interna. Un WSS bajo indica que los componentes dentro de un cluster son muy similares entre sí (alta cohesión).Coeficiente de Silueta (s_i): Es una medida que combina cohesión y separación. Su valor oscila entre -1 y 1:1: El componente está muy bien ubicado en su cluster.0: El componente está en el límite entre dos clusters.-1: El componente probablemente ha sido asignado al grupo incorrecto.2. El Gráfico de Violín como Herramienta de ValidaciónSi bien métricas como el "Método del Codo" nos ayudan a elegir el número óptimo de clusters (k), el gráfico de violín es esencial para la interpretación física de los resultados.Este gráfico combina un diagrama de caja (boxplot) con una curva de densidad de probabilidad. En el contexto de mantenimiento industrial, nos permite visualizar:Distribución y Densidad: Podemos ver dónde se concentra la mayoría de los datos de sensores (vibración, temperatura) para cada grupo.Separación de Grupos: Si los "cuerpos" de los violines de diferentes clusters no se solapan significativamente, el algoritmo ha logrado identificar estados operativos claramente distintos (ej. Estado Normal vs. Falla Inminente).Detección de Anomalías: La extensión de las "colas" del violín permite identificar máquinas que, aunque pertenecen a un grupo, presentan valores extremos que requieren atención.

```{r}
# Librerías necesarias
library(ggplot2)
library(cluster)

# 1. Simulación de datos de sensores (Normalizados)
set.seed(123)
vibracion <- c(rnorm(100, 2, 0.5), rnorm(100, 5, 0.8), rnorm(100, 8, 0.6))
temperatura <- c(rnorm(100, 30, 5), rnorm(100, 50, 7), rnorm(100, 70, 5))
df <- data.frame(vibracion, temperatura)

# Escalamiento (Paso crítico mencionado en tu texto)
df_scaled <- scale(df)

# 2. Ejecución de K-means (k=3)
fit <- kmeans(df_scaled, centers = 3, nstart = 25)
df$cluster <- as.factor(fit$cluster)

# 3. Medición de Performance: Coeficiente de Silueta
sil <- silhouette(fit$cluster, dist(df_scaled))
avg_sil <- mean(sil[, 3])
cat("El coeficiente de silueta promedio es:", avg_sil)

# 4. Visualización con Gráfico de Violín
ggplot(df, aes(x = cluster, y = vibracion, fill = cluster)) +
  geom_violin(trim = FALSE, alpha = 0.7) +
  geom_boxplot(width = 0.1, fill = "white") +
  labs(title = "Distribución de Vibración por Cluster Industrial",
       subtitle = paste("Performance (Silueta):", round(avg_sil, 2)),
       x = "Segmento de Maquinaria (Cluster)",
       y = "Nivel de Vibración") +
  theme_minimal()
```

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

R es un lenguaje muy parecido a matlab que en general pueden ser matrices

```{r}
a=38
a=40
b=60
c=b-a
c
```


```{r cars}
summary(cars)
```

Including Plots

You can also embed plots, for example:

```{r pressure}
plot(pressure, type="l", main='Comportamiento del gas ideal',ylab='hPa',xlab='K')

```

Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

```{r}
Y=40
alpha<-30
```

##uso de datasets o base de datos internos de R

Usando el comando de data en la consola obtenemos distintos datos ya cargados en la base de datos de R, estos datos estan en tablas de las cuales podemos elegir columnas especificas si escribimos $ despues del comando de informacion especifico y elegimos la columna de datos deseada

Comando plot
Es un comando que permite graficar caulquier fuente de datos que le asignemos, teniendo tambien la posibilidad de asignar nombre a las variables o hacer cambios al grafico. En caso de duda de como utilizar el comando podemos escribir *?plot* en la consola, lo cual abriria una ventana que explicara en detalle el comando.

Funciones estadisticas
%explicar despues

```{r}
z1<-rnorm(35,22,5)
z1

```

%350 valores, con un promedio alrededor de 22 con una desviacion estandar de 5

```{r}
hist(z1,main='Histograma de edades',breaks=50)
```

```{r}
density(z1,type="d")
```

```{r}
plot(density(z1))
```

```{r}
## Ejercicio: Operaciones con Matrices (Matriz Inversa)

# 1. Definimos los datos basados en tu DNI (506)
# Usaremos las cifras 5, 0, 6 y combinaciones para completar la matriz 3x3
dni_cifras <- c(5, 0, 6, 6, 5, 0, 1, 2, 3) # Agregamos valores extra para completar 9 elementos

# 2. Creamos la matriz 3x3
# byrow = TRUE organiza los números por filas
mi_matriz <- matrix(dni_cifras, nrow = 3, ncol = 3, byrow = TRUE)

cat("--- Matriz Original (basada en DNI) ---\n")
print(mi_matriz)

# 3. Verificamos el determinante
# Una matriz solo tiene inversa si su determinante es distinto de cero
det_matriz <- det(mi_matriz)

cat("\nDeterminante de la matriz:", det_matriz, "\n")

# 4. Cálculo de la inversa
# Usamos una estructura condicional (if) para verificar la viabilidad
if (det_matriz != 0) {
  matriz_inversa <- solve(mi_matriz)

  cat("\n--- Matriz Inversa ---\n")
  print(matriz_inversa)

  # 5. Verificación (Matriz * Inversa = Identidad)
  cat("\n--- Verificación (Producto debe ser la Identidad) ---\n")
  print(round(mi_matriz %*% matriz_inversa, 2))

} else {
  cat("\nLa matriz no es invertible (el determinante es 0).\n")
}
```